CAUSAL MAP GARDEN

Task 2 – Coding with AI

Contents

Intro – auto-coding with AI

Chapter contents.

15 Oct 2025

Causal coding is fascinating but can take a lot of time. Using AI to help you is pretty easy, but there are still quite a few things to think about, as we will explain in this chapter.

PAGES IN THIS CHAPTER

A workflow for causal coding with and without AI

Contents

29 May 2026

Summary

You have a stack of documents or interviews and you want to answer research or evaluation questions rigorously. This is one workflow for getting there: eight steps, from planning through coding to a final judgement. The steps are almost the same whether you code by hand or with AI. This steps presented in this paper match the way we work in the Causal Map app, but the principles should make sense however coding is done. There is a strong focus here on AI-supported coding at scale, as the scale of AI coding requires some additional procedures and checks, but a manual coder follows approximately the same path and can just skip those sections.

Most subscribers to our App have coded manually, and coding manually is great. But although there's a lot of documentation, we never really did a step-by-step guide to how to do manual coding.

At Causal Map we've also been using AI for causal coding systematically now for nearly four years in a set of really interesting studies, mostly for clients, and often at considerable scale. Plenty of subscribers have been asking to use AI themselves. We've been reluctant frankly because we've been making it up as we go along and there are a *lot* of different things to think about. But now we've introduced One-Click Coding and it's time that we spilled out something of what we have learned on AI coding for the benefit of others.

So that is two overlapping reasons for this working paper. It is written so that a manual coder can read straight past the AI-only parts (the AI decisions table, and the model, chunk and iteration parts of Step 4) and still have a complete workflow.

The steps can be divided into three Tasks. **Collect** (Steps 1 to 2): decide what questions you want to answer and gather data that can answer them. **Code** (Steps 3 to 5): turn the text into a checked table of many causal claims, each with a quote and a source. **Query** (Steps 6 to 8): weigh that evidence and use it to answer the questions. The pattern is one or more cheap, wide coding passes to capture the evidence, then steadily narrower judgement, so a thousand raw claims might end as a few dozen well-vouched links and a few strong findings.

The companion piece, Quality assurance at each step, goes through the same steps and asks how to keep each one rigorous. For step-by-step app instructions in the Causal Map itself, see AI coding.

About causal mapping

Causal mapping analyses what people say in interviews, focus groups or reports when you want to know what they think causes what. You read the material and code each causal claim ("the rains ruined the harvest", "the training raised her confidence") as a link from one factor to another. Combine the links from many sources and you have a causal map: a network of what people believe drives what. For a fuller introduction see this and this.

It is like systems mapping, but instead of modelling how the world works we first record what people claim about it, and only later, if at all, ask what is really going on.

We code in the **minimalist** style: a link records only that "a source says X influenced Y", with a quote. No polarity, no strength, no fitted curves, no counterfactual the speaker never gave. The case for that is in Minimalist coding for causal mapping.

The eight steps

1. Collect: Overall planning: questions, methods
2. Collect: Gather data
3. Code: Prepare and revise the codebook
4. Code: Code the claims
5. Code: Check links and iterate
6. Query: From claims to bundles
7. Query: From bundles to pathways
8. Query: Judge value, contribution and the final judgement

The steps are not a strict sequence. Sometimes you will iterate. You will revisit the early ones as results come in, and only the last is strictly required; most projects use a handful.

Step 1: Collect

Start from the question. Before anything else, write down what you want to be able to say at the end, and to whom. Everything downstream, the data you gather, the labels you allow, the columns you add, the queries you run, follows from that.

Be realistic about what causal mapping can and should answer. It is good at: which factors matter most, what influences or follows from a given factor, how different groups see things, how well the evidence supports a pathway or a theory of change, and the overall structure of the system. It will not give you effect sizes, and on its own it does not prove that X causes Y; that judgement stays with you (see Quality assurance at each step of the causal coding workflow). So pick questions the method can serve, and only as many as the evaluation needs. The menu of question types is in the questions chapter.

It helps to sketch, before you code, the map or table that would answer your question: which factors, which comparison, which pathway. That sketch is your target.

Treat the question as a first draft. Causal mapping is partly exploratory, so expect to sharpen it once early coding shows you what the sources actually talk about.

Causal Map's AI assistant, MapCat, can help walk you through these decisions, design and iterate on a corresponding coding plan, and then build a report to answer those questions.

How this fits the wider field

Causal mapping is rarely the whole of your research or evaluation project. It is an evidence broker: it gathers and organises causal claims as part of your process of making evaluative judgements which may include other approaches. You can see it as belong within the causal pathways family of methods, or perhaps as a tool which feeds into those methods: contribution analysis (Mayne n.d.), process tracing (Befani & {Stedman-Bryce n.d.; Collier n.d.), Outcome Harvesting ({Wilson-Grau n.d.; Britt et al. 2025), realist evaluation (Pawson & Tilley 2013), QuIP (Copestake et al. n.d.) and Most Significant Change (Davies & Dart n.d.). Most real evaluations combine several such methods, what Apgar and Aston call bricolage: you pick and combine the methods to fit the question (Apgar & Aston n.d.; Apgar 2024). The nine steps here map onto the four stages they describe for a causal pathways evaluation: design and questions (Steps 1 to 2), methods and data (Step 2), causal analysis (Steps 3 to 7) and assessing the strength of evidence (Steps 6 to 8).

Step 2: Gather data

The question decides the data. Work out which sources you need, covering what, so the comparisons you care about are possible later. If you will want to compare women and men, or staff and clients, or early and late, those groups have to be in the data and recorded in the source metadata, which Step 5 and the query steps lean on.

Narrative material works best: ask people what changed and why, and you get causal claims to code. QuIP-style "stories of change" are gathered in exactly this way (Copestake et al. n.d.).

Gathering data is a subject all of its own and we only touch it here; this focus of this paper is coding and analysis. See .

Step 3: Prepare and revise the codebook

This step is the same whether you or an AI does the coding: you decide how tightly the labels are fixed in advance, and you organise and revise them as the work goes on. With AI the choice is an instruction; by hand it is your own discipline, but the trade-offs are identical.

You can start from nothing (free coding), from a fixed codebook such as a theory of change, or somewhere between, and you will often revise it more than once.

How free should it be? Four common choices (read "the model" as "you or the model"):

- **Forced**: only your labels; anything else is dropped.
- **Mostly fixed**: your labels, but let the model add new ones, flagged (for example `[new]`) for review.
- **Hierarchical compromise**: fix the top level, let the model fill in the detail (see Hierarchical coding).
- **Free**: the model invents everything.

Loose coding finds more but leaves more to tidy; tight coding is cleaner but misses links. If you allow too many off-codebook labels you face a lot of recoding; if you allow too few, your maps thin out and you wonder why you bothered finding the links at all.

Recoding

Recoding is how you revise the codebook after a first run, which is why it belongs here. Whether you coded by hand or with AI, free coding leaves you with many overlapping labels, and consolidating them is the same job either way (see Different kinds of coding and recoding):

- **Hard recode**: rewrite the codebook and code again. Most work, best results.
- **Links or factors recode**: clean up label by label. By hand, edit in the Links or Factors table, use search and replace, or use Bulk Edit; with AI, use AI Answers.
- **Soft recode**: cluster or magnetise labels into a smaller set.

For organising a large codebook, deciding on a labels-plus-tags system, and bulk rewriting, the recoding paper Different kinds of coding and recoding is the detail; the same tools serve manual and AI coding.

Step 4: Code the claims

By hand, coding means reading the text, highlighting each causal claim, and recording it as a link from one factor to another with its quote and source. The decisions that follow (labels, hierarchy, when to add a column) apply just as much to manual coding; the rest of this step is the AI mechanics for doing the same thing at scale. For a hands-on first manual project see Manually code your first project.

With AI, coding means writing an instruction, much like a chatbot prompt, that you paste into the app. It tells the app the context and what labels and columns you want. You do not need to add the text itself; the app does that.

In a hurry, or coding just one short text? Press **One-click**: the app codes with all defaults (claim by claim, no codebook), chunks long texts for you, and joins up the resulting fragments afterwards. Or ask **MapCat**, the assistant inside the app, which conducts the set-up conversation for you and reports every choice it makes. Both run the same orchestration machinery, described at How the app orchestrates an AI run below. Often that is enough. The rest of this step is for when you want control.

The golden rule: test your instruction on a small, varied sample, work out exactly why the output is wrong or thin, change it, and run again, until you are happy. Then scale up.

Holistic or claim by claim

Currently not implemented

Currently we do not use holistic coding in the app, as we found that although it tells a coherent story, this is at the expense of some of the links not really being causal. We may re-introduce it in the future. Instead we use `join-islands` processing to get the bigger picture.

Holistic coding asks the model for one connected diagram per chunk. You get a cleaner, joined-up story, best for a single short text, but the model has more freedom over what to include. (Oddly, asking for a diagram yields better-connected networks than asking for a list of links; under the hood we ask for a *diagram* and convert it.) Claim-by-claim coding asks for every link separately. You get fuller coverage, better for many texts, but the links join up less and you rely on recoding to rejoin chains: if the text says A to B to C to D and the model codes A to B and C to D with slightly different middles, a later recode has to spot that they match.

Dimension	Holistic	Claim-by-claim
Main aim	One connected network	Every link in the text
AI freedom	More: it picks the story	Less: it just finds links
Connectedness	Higher	Lower; needs recoding to rejoin chains
Best for	One short text	Many texts, or exhaustive coverage
Recall	Often secondary	Usually as important as precision
Repeated claims	May miss them	More likely to catch them
Quotes	One per link	One per link

Holistic versus claim-by-claim coding

Chunk size and sampling

The more text you give the model at once, the thinner its coding: one page can yield as many links as five. In our experiment series (synthesised here) chunk size mattered more than anything we did to the prompt wording: chunks of 2,000 characters found half as many links again as chunks of 4,000, and a whole 89,000-character document sent in one request returned 6 links where chunked runs returned over 100.

The reason is that models **satisfice**: given a long stretch of text, they report a plausible handful of claims and stop, whatever the instruction says. But small chunks have their own cost: a cause discussed early in a document and its effect discussed pages later never appear in the same chunk, so the coded map arrives fragmented. The app's orchestration counters both problems: **segment accounting** obliges the model to account for every paragraph, so large chunks behave more like small ones, and a **join-islands** pass hunts afterwards for the missing cross-chunk connections, demanding a verified quote for each. Both are described under How the app orchestrates an AI run below, and MapCat, the app's assistant, will choose these settings for you if you tell it whether you want fine detail or broad strokes.

Even with those remedies the trade-off remains real. In our tests (single runs), a plain run at 16,000 characters kept only around a seventh to a third of the links a 2,000-character run found; adding segment accounting brought that back to roughly half to three quarters, depending on the text, at an eighth of the number of requests. Larger chunks are quicker, cheaper and better at catching connections that span pages; smaller chunks still win on sheer recall. When every link matters, use small chunks.

On a big corpus, sample first: with 1000 pages, code 100, review, code another 300, and if it holds up finish the rest. Make the sample random, or stratified by the groups you care about, so you do not tune to one untypical slice (see selecting random samples).

Model

Bigger or newer is not always better. Gemini Flash is the default and often enough. More capable models with larger context windows should do better on bigger chunks, though we have not tested that formally.

Always ask for quotes

Insist on a verbatim quote for every link. Without it you cannot show your working, and the result is not something you could defend as evidence. The app does not enforce this, so put it in your instruction.

Labels

Decide how labels should read: close to the text (in vivo) or more abstract ("talk like a social scientist"). For client-readable, recode-friendly labels a semi-quantitative house style helps, such as "more income" or "lack of resources" (see this). Two devices earn their keep:

- **Tags**: bracketed text on a label, such as `patients (before surgery)`, to build labels from parts.
- **Hierarchical labels** with a separator, which let you zoom out later (see Hierarchical coding).

For coding opposites and sentiment, see Opposites and sentiment in AI coding.

Custom columns

Besides the fixed columns (cause, effect, source, quote), you can ask for custom columns: any attribute you can code consistently across links, such as sentiment. (Tags describe factors; columns describe links.) Each extra column costs you some precision or recall, so keep them off the main pass and add them in a later iteration if they matter. Quality columns for checking links, such as conviction and strength, come in Step 5. More on columns: Adding and using custom columns for your links.

When you free-code without a codebook, a sentiment column is worth adding: the app groups labels by meaning, and "less X" lands right next to "more X", so a sentiment column keeps them apart.

Context and named entities

Give the model enough background to know the job: the project, the audience, and the names, abbreviations and preferred phrasings specific to your work, so it settles on one consistent label where the text uses several. Keep it short; more than a page of context can start to cost you recall (see You have to tell the AI what game we are playing right now).

Iterations

You can run extra passes over the same text (separated by `====`; the app handles this). Use them to check accuracy, mop up missed links, or add a column. Each pass roughly multiplies time and cost, and a better first instruction usually beats a second pass. A follow-up might say:

```
Check for mistakes and correct them.
Delete links with too little evidence, or where you
assumed a cause or effect.
```

Only the final pass feeds the app.

How the app orchestrates an AI run

You can use One-click coding without knowing any of this, but if you are going to rely on AI coding it helps to know what happens between pressing the button and seeing the map. The same machinery runs whether you choose the settings yourself or let the app's assistant choose them for you. And the disciplines it applies, exhaustive accounting, verbatim quotes, every decision on the record, are worth borrowing even if you never touch AI at all.

MapCat, the optional conductor. MapCat is the chat assistant inside the app. In guided mode it conducts the set-up conversation a methodologist colleague would: what do you want to find out, how fine-grained should the coding be, whether cost or quality matters more, whether factor labels should name the actors or stay in the source's own words, whether to use a hierarchical codebook, and how far to go in joining up the map afterwards. It turns your answers into settings and starts the run. Two guarantees underpin this. Every decision it takes on your behalf appears on a decision card in the chat before anything runs, and the same decisions are recorded exactly in the run logs, so another analyst can see what was done and repeat it. MapCat is an optional layer: everything it sets, you can set by hand in the coding panel. We describe it here because it gathers into one conversation the decisions that any AI coding has to settle, with or without an assistant.

Segment accounting, the cure for satisficing. Left to itself, a language model reads a page, reports the three or four most striking causal claims, and stops. Researchers call this satisficing: doing just enough to produce a plausible answer. It is the main reason naive AI coding misses so much, and no amount of pleading in the instruction fixes it. What fixes it is bookkeeping. Before coding, the app splits the text internally into short numbered segments, and tells the model to return a verdict *for every numbered segment*: either the causal claims it found there, or an explicit entry saying why that segment has none, quoting its most causal-sounding phrase as justification. A model that has to account for every segment cannot skim. The rule cuts the other way too: a justified "nothing here" beats an invented claim, so the accounting raises recall without licensing fabrication. With this in place we can give the app chunks of *eight times the size*, several pages at a time, and still find roughly half to three quarters of the links that small chunks would (see the experiments synthesis). This is several times quicker and cheaper, and it reduces the "window effect": the app has more chance to notice links which span larger sections of text.

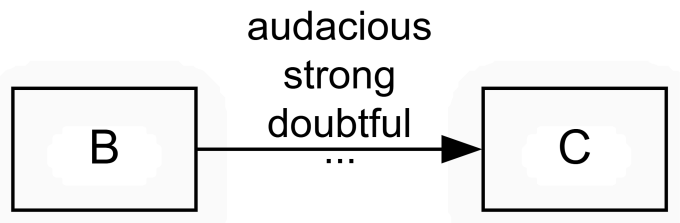
Joining the islands. Coding chunk by chunk has a side effect: the same factor gets slightly different labels in different chunks, and links between distant parts of the text are missed, so the finished map arrives as an archipelago of small unconnected islands. An optional join-islands pass then re-reads the whole text alongside the coded links and a numbered list of the islands, and must return a verdict for every island, the same accounting discipline again: merge a label with one elsewhere that names the same factor, propose a connecting claim with a verbatim quote, or state that the text really does leave that island unconnected. The app then checks every proposed quote mechanically: its fragments must appear word for word in the text, in order, or the link is discarded and the discard is reported. Rounds repeat until one finds nothing new. On one test text, a 28,000-character account of the outbreak of the First World War, the pass took the map from 59 separate islands to 6, with every added link's quote machine-verified.

The pass has two modes, because it embodies a methodological choice. **Merge** consolidates labels and never adds a link, so nothing appears in the map that the coding did not find; choose it when you want strict reproducibility, or when you plan to consolidate labels across the whole project with a recode anyway. **Full** also adds the quote-checked links between islands. The result is far better connected, and every added link carries a verified quote, but these are still the least certain links in the map: a quote can check out while the reading of it is subtly wrong. Full mode suits a presentation map, or a single rich source where you want each account to hang together; merge mode suits work headed for publication. Either way the added links carry a marker recording where they came from, so you can review or exclude them at any point.

Why we show the workings. This is our general approach to orchestrating AI, and MapCat simply makes it visible: split the work into small pieces the model must account for one by one, verify mechanically whatever can be verified (quotes above all), report what was dropped as well as what was kept, and record every decision so the run can be repeated. Even the coding instruction is built this way: a minimalist core saying what a causal claim is, plus separate optional layers for hierarchy, opposites, extra columns and the codebook, following Minimalist coding for causal mapping. Each convention is a choice you make on purpose rather than a default you inherit. A manual coder can follow the same disciplines by hand: account for every page, insist on verbatim quotes, and write down each convention you adopt.

Step 5: Check links and iterate

However careful the coding, some links will be wrong, so check and enrich them before you analyse. You will often see bundles: several links between the same cause and effect, from different sources or different parts of one source (see Bundle of Links — definition).



Start by tagging. A free tag such as `#doubtful` or `#surprising` records a misgiving you can filter on later.

Then add columns if they help:

- **Conviction**: how sure the source sounds (weak, neutral, strong). Most claims are unmarked, so most are neutral. This records how confident the source is; it does not measure the strength of the link.
- **Strength**: whether the source explicitly calls the influence strong or weak. Again, usually neutral.

Do not read these as scores like 1, 2, 3: neutral means "not mentioned", not "medium". That most people never mention strength does not mean they think it is medium; often the idea simply does not apply (on why we resist coding strength, see Our approach is minimalist — we do not code the strength of a link).

You can also score sources rather than links, for example reliability or role. Because every link has a source, those scores reach every link for filtering.

Steps 6 to 8: querying the model

Your links are a queryable knowledge graph

Your coded, checked links are not a static report; they are a model you can query, over and over, to answer different questions (see Causal mapping produces models you can query to answer questions). The links table is a **knowledge graph**: a network of factors joined by one kind of relation, "influences". Because the relation is always causal, a lot of evaluation questions can be answered almost out of the box, which is what a general-purpose knowledge graph cannot do (see Causal maps are knowledge graphs, but with wings).

The way you query it is with **filters**, and the key idea is that **a filter is a question** and **filters chain**. Each filter takes the links table and narrows or rewrites it; stack several and you build up the answer to a sophisticated question, where order usually matters (see Combining questions). It helps to picture the analysis as a pipeline, the links table passed through one transform after another:

```
Links |> filter to women only |> trace paths from training to
income |> zoom to top level |> bundle |> show map
```

The same dataset yields very different maps with no contradiction, because each map is just the result of a different chain of filters. The minimalist coding paper develops this pipeline view in full (see Minimalist coding for causal mapping).

What you can ask

Many questions answer themselves the moment coding is done, straight off the links. Some examples, each with its own page in the questions chapter:

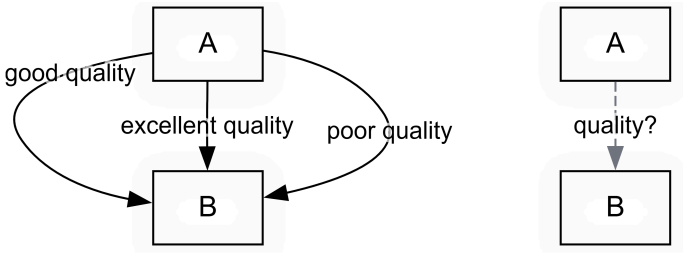
- Which factors and links are mentioned most often or by the most sources?
- What are the main outcomes and main drivers?
- What leads to or follows from a factor you care about?
- How do groups differ, and are there hidden subgroups?
- What is surprising or emerging?
- What is the overall structure of the system, and are there feedback loops?

The three steps that follow are for the harder questions that need defensible answers, where you weigh the evidence rather than just count it. Here is where each kind lands.

Question	Where
Top factors, drivers and outcomes; what influences or follows from X; group differences; surprises; network structure	straight off the links, from Step 5
How robust is the evidence that X influences Y?	Step 6
Pathways from X to Y, indirect ones included, without the transitivity trap	Step 7
; rival explanations;	Step 8
A ; does the whole thing hold together?	Steps 7 and 8

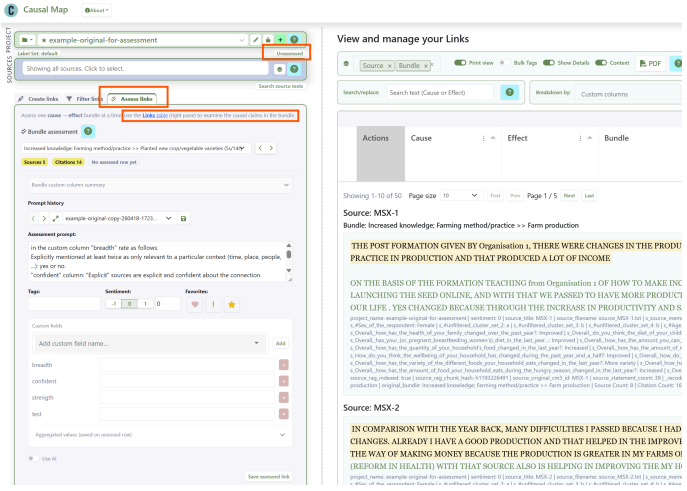
Step 6: From claims to bundles

A bundle is the set of claims that all say the same X influences Y, from different sources or different parts of one source. Whatever else you do, weighing each bundle as a whole is part of quality assurance: how many sources, how convincing, do they agree or pull apart? Always look at your bundles this way before you build on them. This step has its own paper: Assessing quality or robustness of evidence for a causal link based on a bundle of coterminal causal claims, and see also here.



Once coding and cleaning are done, decide which bundles you will take seriously: the ones that survive your filters, perhaps after zooming to a higher level or restricting to certain sources. There might be five or a hundred. This is the evidence the rest of your analysis rests on.

You can stop there, having weighed the bundles by eye. Or you can record that judgement formally. Causal Map has a newer, optional feature that collapses a bundle into a single **assessed link** between the two factors, carrying your quality scores and, by default, the bundle's citation and source counts. The underlying claims are not deleted: a switch shows either the assessed links or the unassessed bundles, never both at once. Thin bundles can yield no assessed link, or one marked "Passed? = Fail".



Creating an assessed link from a bundle, bundle by bundle, in the Causal Map app

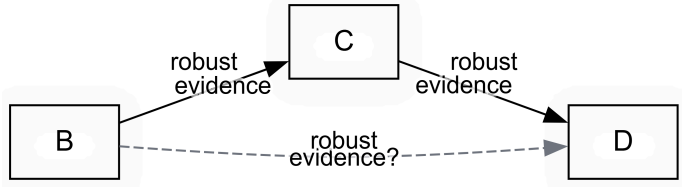
You can do this by hand, or let the AI take a first pass against your rubric and review it. The app will not create assessed links until you have written that rubric down, on purpose. The rubric can be a yes/no, a 1-to-5 scale like the one in Jewlya Lynn's seafood retrospective (Lynn 2026), or several dimensions such as confidence and triangulation.

Either way, formal or by eye, the move is the same: from a mass of raw claims to a smaller set you are willing to vouch for. A project might go from 1000 raw claims to 30 bundles to 25 assessed links, a much cleaner basis for argument.

Step 7: From bundles to pathways

This step is about queries to answer more specific and sophisticated questions and is potentially also a move to causal inference.

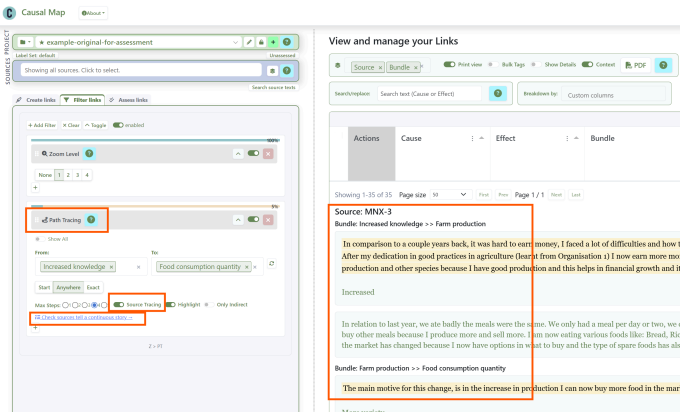
Now you can ask about pathways, often indirect, from an intervention to an outcome.



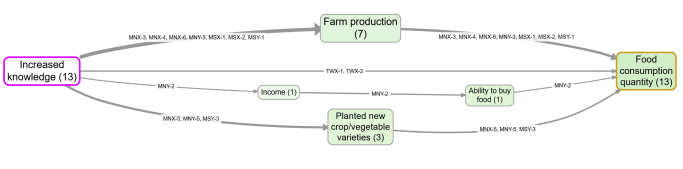
Even with every link well grounded you are not done, because conclusions usually run across a web of indirect links, from B1 and B2 to C via E, F and G. Two tools help.

Path tracing keeps only the links on some route between your chosen start and end factors, within a set number of steps (see Path tracing and source tracing).

But "A influenced B" and "B influenced C" does not give you "A influenced C": the contexts may not overlap. This is The transitivity trap, the biggest pitfall of any causal diagram, and the heart of the companion QA paper. **Source tracing** is the safe move: it keeps only sources whose own account runs all the way from A to C, so every link belongs to at least one complete story and you can review the evidence source by source.



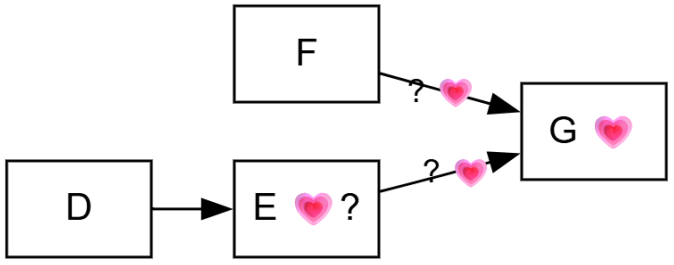
Setting up source tracing from Increased Knowledge to Food Consumption Quantity, and reading the narratives.



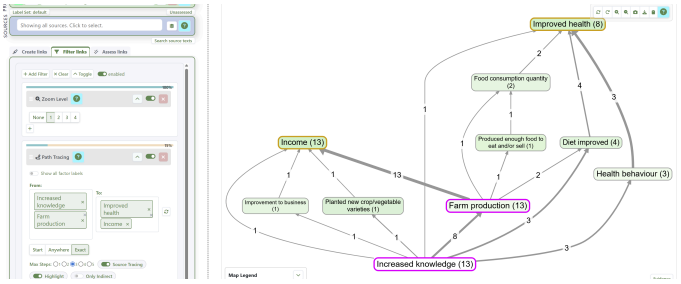
The matching map, here showing source IDs and counts for easy checking.

If you have assessed your bundles, you can trace on the assessed links (clean counts, no quotes) or the raw ones (quotes, busier map); often you will want both.

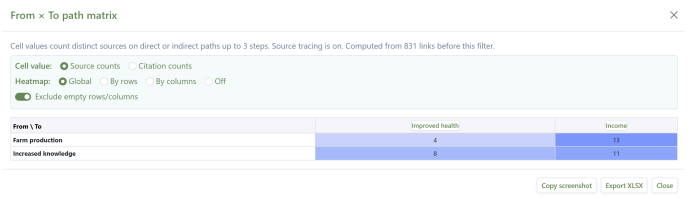
Step 8: Judge value, contribution and the final judgement



Judging how much something mattered, and weighing it against rival explanations, is central to evaluation and well covered elsewhere, not least by John Mayne (n.d.); QuIP has much to say on value (see Powell (n.d.)). The discipline is to compare your influence against the alternatives on the same map, not in isolation. For counting and comparing influences with path and source tracing, see Counting and comparing influences. For example, tracing the single-source narratives from two drivers to two outcomes:



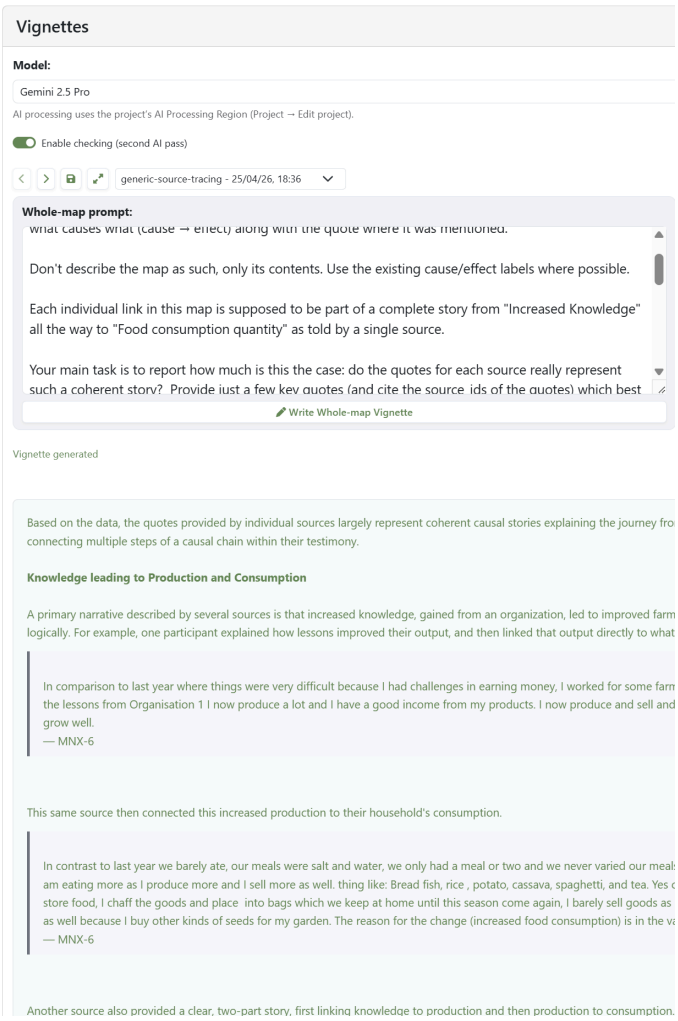
and counting the sources with a complete narrative between them:



Holistic final judgement

Finally, draw the conclusion. You have checked the claims, assessed the bundles, traced the pathways and weighed the alternatives; now look at all the evidence at once and decide. Behind a single map there may still be hundreds of quotes. Does the claim hold up? Do all the links really belong to the same context?

The AI vignette feature helps: it drafts a commentary on a view, drawing on the underlying paths, links, quotes and source data, and can answer set questions, for example "is each link part of one coherent story from intervention to outcome?".



An automated vignette tasked with checking whether the evidence for each pathway is coherent.

A common use is a source-by-source commentary on the pathways from an intervention to an outcome, judging how coherent each account is. The AI does only what a patient reader could do with the same quotes, so treat its draft as a starting point and edit it.

Then close the loop: does the evidence answer the question you set in Step 1?

References

Apgar (2024). *A PARTICIPATORY APPROACH TO EXPLORING CAUSAL PATHWAYS Experience from the CLARISSA Programme July 2024*.

Apgar, & Aston (n.d.). *How Do We Define and Support Quality and Rigor in Causal Pathways Evaluation?*.

Befani, & {Stedman-Bryce (n.d.). *Process Tracing and Bayesian Updating for Impact Evaluation*. Evaluation, *23*, 42--60.

Britt, Powell, & Cabral (2025). *Strengthening Outcome Harvesting with AI-assisted Causal Mapping*.

Collier (n.d.). *Understanding Process Tracing*. PS - Political Science and Politics, *44*, 823--830. https://doi.org/10.1017/S1049096511001429.

Copestake, Morsink, & Remnant (n.d.). *Attributing Development Impact: The Qualitative Impact Protocol Case Book*. March 21, Online.

Davies, & Dart (n.d.). *The Most Significant Change Technique, A Guide to Its Use*.

Lynn (2026). *HU Seafood Retrospective*.

Mayne (n.d.). *Assessing the Relative Importance of Causal Factors*.

Mayne (n.d.). *Making Causal Claims*. ILAC Brief, *26*.

Pawson, & Tilley (2013). *Realistic Evaluation*. Sage Publications Limited.

Powell (n.d.). *Theories of Change: Making Value Explicit*. Journal of MultiDisciplinary Evaluation, *15*, 53--54.

{Wilson-Grau (n.d.). *Outcome Harvesting: Principles, Steps, and Evaluation Applications*. IAP.

🌻 Different kinds of coding and recoding

Assuming you're not coded with a fixed codebook, and assuming you're coding multiple sources or you've got texts which are longer than your selected chunk size, so you are breaking them up into multiple chunks, then you can expect to end up with very many labels, many of which overlap in meaning.

You can address this either with

- soft recoding or magnetic relabelling
- hard recoding: once we've done a fair amount of coding, or all the coding, we then group those labels either pre-clustering them using embeddings or and or applying an AI and or putting a human in the loop to get a list of labels for hard coding, or rather a system because you might have a smaller set of labels and additional tags or columns.

We haven't done the research to find out whether having say 10 labels and three tags is more or less efficient than the corresponding set of 60 labels.

And of course there's also the newer possibility of using the AI in the AI Answers feature to take an existing large set of coded labels and then recode them into a more compact set. This has also been discussed here Different kinds of coding and recoding

So you've basically got a lot of decisions to take as you work your way through the coding pathway. And it all depends on lots of different things like how long your text is, how many different documents you have. Oh, I didn't mention that in Causal Map we never, we do break down long source texts into smaller chunks but we never combine smaller source texts into larger chunks, so each source is always coded on its own. So if you don't have a fixed codebook then you'll always get many labels with overlapping meanings.

	Hard coding	**Hard recoding**	**Links recoding** (**Causal** in the app)	**Factors recoding** (**Semantic** in the app)	**Soft recoding**
Accuracy	Highest	…	…	…	Lowest
Speed	Slowest	…	…	…	Fastest

	Hard coding	Hard recoding	Links recoding (**Causal** in the app)	Factors recoding (**Semantic** in the app)	Soft recoding
Manual	Just code manually	Make a copy of your file, delete links and start again	Edit manually in Links table or Map, - or use search/replace in Links table	Edit manually in Factors table or Map, - or use search/replace in Factors table - or Bulk Edit	-
AI	Just code with AI, with/without a codebook	As above, or just put the switch "skip coded sources" to off	AI Answers / Links. Writes into whichever Label Set is active in the toolbar.	AI Answers / Factors. Writes into whichever Label Set is active in the toolbar.	Apply magnetic labels in Soft Recode filter

What's the point of Links and Factors recoding? What's the difference?

The Recoding buttons in the app say **Causal** for links recoding and **Semantic** for factors recoding, because that is the difference: Causal reads each link whole, cause and effect and quote, and matches by the claim being made, while Semantic reads a factor label on its own and matches it by what the words mean. Magnetic is the same judgement as Semantic done by embedding rather than by reading. The buttons were called "AI links" and "AI factors" until August 2026.

- Soft recoding is only as good as the underlying embedding space, and it is never perfect.
- Hard recoding can take a long time, is expensive, and does not encourage experimentation
- With Links/Factors recoding, you can:
  - Recode just the currently filtered sources/links (or all links)
  - Recode into whichever Label Set is active. Pick `default` in the Label Set widget (toolbar, below the Sources bar) to write to the permanent cause/effect labels; create a named set such as `experiment1` and the AI writes to `cause_experiment1` / `effect_experiment1` instead. Flip between sets in the same widget to view either.
  - There is also another option Answers which is not about recoding; it is simply a way to send your links and/or factors data to an AI and getting a text answer.

But the main point is that rather than just hoping the magnetisation will work the way you want it to, you can do smart recoding as if you had an assistant to work through each label. For example you can say "Relabel everything which expresses a decrease or lack of something with a ~" or "Look at all these labels and tag each with `[Food]` or `[Health]"

.

- You can even bring other columns into play, for example citation count, source count etc.
- Links recoding is significantly more powerful because you can also include the actual Quote as well as both Cause and Effect. This means the AI can make its decision with a lot more context. So this is almost like recoding from scratch, but the original coding has already identified causal claims and all we have to do is relabel the labels with the same complete information about the claim.
- Be careful: it's tempting to say things like "Find 3-8 top-level factor labels which cover the meaning of all these labels and recode them with the new top-level labels", but remember the "Rows per call" slider: with a large set of links and lots of quotes you will probably have to break up your work into multiple chunks, and each call may come up with different labels. In this case you could use the Answers mode (or the Cluster part of Soft Recode filter) first to develop some labels.

AI CODING EXPERIMENTS SYNTHESIS

Coding texts with LLMs: some transferable learnings from a recent experiment series

Causal Map is software for a particular kind of qualitative research: you feed it interviews or documents, and it helps you map out the causal claims people make, that is, statements that one thing affected another ("the training improved my confidence", "the funding cuts meant we lost staff"). Its central AI task is therefore a large coding job: give a language model a long, messy transcript and ask it to return a list of structured items, each one a causal claim with a cause, an effect and a verbatim quote to back it up. There is no single right answer, the items involve judgement, and quality can only really be checked by a person reading each item against the source. In mid July 2026 we spent a week running about 34 numbered experiments on this task: 22 prompt versions, 9 models, 4 chunk sizes, several multi-model pipelines and three ways of judging output quality. The full log is 1,900 lines (`causal-map-extension/docs/plans/ai-coding-prompt-experiments...md`). This note is the distillation, written for readers with no special interest in causal mapping, because almost none of what we learned is specific to it. If your task is "get an LLM to pull many judgement-laden items out of long text", whether that is entity extraction, qualitative coding, screening studies for a systematic review or annotation at scale, most of this should transfer.

A few words that recur below. **Recall** means how much of the real content the model found; **precision** means how much of what it returned was right. A **chunk** is a slice of the source text sent to the model in one request, because long texts are usually processed in pieces. A **link** is our unit of output, one extracted claim of the form "this led to that". Because the effect of one claim is often the cause of another ("cuts led to stress" and "stress led to sickness" share "stress"), the links join up into a network, which is the map the user actually wants; a link that connects to nothing else is an **island**. None of the lessons below depend on this network aspect except where it is spelled out.

Key learnings

Measurement

1. **An LLM judge is a relative signal only. Human and LLM judges need to work together.** Reading hundreds of items by hand is slow, so like many teams we used a second LLM as a judge, asking it to rule on each extracted item. When we finally audited the judge against a human, it ran about 20 points optimistic and agreed with the human on "is this item usable?" just 63% of the time. Yet it also beat the human in places, catching an inverted cause-and-effect the human missed. So neither is a gold standard: an LLM judge can rank prompt A against prompt B, but its absolute

numbers mean little. Hand-check a real sample before shipping any conclusion; a 60-item audit overturned our headline result.

2. **We rejected a gold standard altogether.** The classic way to evaluate extraction is a gold standard: a hand-made list of the correct answers, and you score the model against it. For judgement-laden tasks this collapses, because two competent human coders produce different but equally valid extractions. Only the wrong items are certain, and they cannot be listed in advance. The workable instrument turned out to be a **verdict bank**: a stratified sample of real model outputs, each ruled OK or NOT by a human, against which any automated judge can then be scored.

3. **Run repetitions, even at temperature 0.** Temperature 0 is the setting that is supposed to make a model's output near-deterministic. It does not make it repeatable enough to trust one run: identical prompts on identical text returned counts of 4, 1, 0, 0 across four runs for the feature we were tuning. That spread covered the entire range our prompt variants differed by, so a week of single-run comparisons had measured noise. Treat any single-run gap under about 10 points as nothing, and re-run before believing anything.

Silent data loss

1. **Most "this model is bad" results were pipeline bugs.** Between your prompt and the model there is plumbing: API calls, retries, parsers, output files. Ours failed silently in seven distinct ways, and every one made a model look worse than it was: reading only the first part of a multi-part response; rate-limit errors ("429", the API saying "too busy, try again") dropped without retry; responses cut off mid-answer but flagged as complete; correct answers wrapped in markdown formatting and rejected by a validity check; runs overwriting each other's output files; an expired security token silently reused. The trap is that a run which lost half its data reads exactly like a cautious model. Before believing any run, confirm that every chunk actually came back.

2. **A fix to the test rig is only half a fix.** We fixed this missing-retry bug in our experimental harness, and the identical bug stayed live in the product for two more days, until a real user's transcript came back with 2 items instead of 40 and they concluded the AI was useless. Every time you fix the measuring tool, ask whether the product fails the same way.

Prompts

1. **Shorter prompts performed better, down to a floor.** Our prompt had grown the way prompts do: rules, exceptions, worked examples, warnings. Cutting it from 10,500 characters to 7,000, keeping every rule but trimming the prose and the examples, improved recall and precision together. Cutting further to 4,000 went too far: recall dropped 20% and the rules compressed to a phrase stopped protecting. Best guess at the mechanism: a shorter prompt keeps the model's attention on the text rather than on rule recitation, provided every rule still appears once.

2. **One prompt for all models.** It is tempting to keep a tuned prompt per model, and when two models behave differently on the same prompt it looks like proof that you need to. In our case the real cause was a prompt that contradicted itself: a newly added section said restatement was acceptable while an older section still banned it, one model obeyed each branch, and we nearly shipped "these two models need different prompts", which was false. A per-model prompt also breaks the day that model retires. When you change a rule, search the whole prompt for the old version.

3. **Wording is a weak lever; step-by-step task framing is a strong one.** We wanted the model to also catch a rare class of item (claims that an influence was attempted and failed, for example "we ran the campaign but nothing changed"). Four rounds of rewording the main prompt, up to and including a full rewrite of the task definition, moved the count not at all. What worked was changing the task: a dedicated second pass over the text asking only for that class found nearly all of them, and stayed quiet on a text that contained none.

4. **Naming a bad pattern can, perversely, teach it.** We added a rule banning one specific bad output form, complete with an example of the form. The frequency of that form went up, from 3 runs in 6 to 5 in 6. Showing the model what not to do put the pattern in front of it.

5. **Models satisfice on sweep tasks.** Asked to "go back over the output and find what you missed", the model returned 1 to 3 items per call, apparently because nothing obliged it to do more than plausibly comply. Restructuring the same request as exhaustive accounting, where every island in the output must either get an answer or be explicitly marked "none", made the same model return 15 in one round. When a sweep returns little, suspect the contract you set before suspecting the model.

6. **The same trick busts satisficing on the main task, which is where it pays.** The same model satisfices on the main extraction too, and this is why big chunks under-code (point 18): over a large context it returns a plausible handful and stops. The fix is the accounting contract from point 10, applied one level down. Divide each chunk into numbered segments (roughly a paragraph each) and require the model to account for every one: at least two claims from each segment where they exist, or an explicit "none". This lifted recall by 20 to 40% at moderate chunk sizes, by three to four times at the whole-document extreme where a plain pass collapses to a handful of items, and roughly halved the wasted effort. The decisive detail is that the escape must be **costly**: an easy "none" gets rubber-stamped without reading, so we require the model to quote the segment's most causal-sounding phrase and say in a few words why it is not codeable. A free skip is abused; a skip that has to be argued for gets the segment read. This softens the hard chunk-size tradeoff of point 18 without removing it: in our runs a plain 16,000-character chunk kept between a seventh and a third of the links a 2,000-character run found, and adding per-segment accounting brought that back to roughly half to three quarters, depending on the text, at an eighth of the requests. The accounting makes big chunks usable; small chunks still won on raw recall in every comparison. Two cautions. It has a ceiling: past about 32,000 characters the accounting itself degenerates into rubber-stamped "none"s, because the segment list grows longer than the model will genuinely read. And on clean prose it can

be neutral or slightly negative, because a short careful paragraph really does hold one claim, not two, so the floor pushes at nothing. It earns its keep on messy, claim-dense material and on large chunks.

Models (July 2026, Gemini and Claude on Vertex AI, EU residency required)

1. **Models differ on multiple dimensions.** The pattern was consistent by model rather than by size or cost. gemini-2.5-flash: high recall, 17% wrong. gemini-3.1-pro, the expensive one: only 2% wrong but it missed over half the real content, and its restraint was avoidance of the faint or debatable claims rather than better selection. gemini-3.5-flash: the surprise, cheap and precise but finding a third of the volume, and our eventual default. gemini-3.1-flash-lite: cheapest by far, quality swinging from text to text. claude-sonnet-4-6 and claude-haiku-4-5: two to three times the error rate of the others on this task, and ruled out.

2. ***Over-produce and prune* beats *under-produce and pad*.** Two ways to combine a generous model and a careful one. The design that worked: let a cheap high-recall model produce too much, then have a more careful model prune, which kept ~95% of the good items and removed ~70% of the wrong ones. The reverse design, asking the careful model to code and then padding its output with a "find more" pass, bought recall by lowering its bar and added errors. And the pruner must be a different model: pruning with the same model that coded left twice the errors in place, presumably because it likes its own work.

3. **But reverse the roles and the same pairing fails: a candidate list makes a strong model worse and dearer.** This sounds like a contradiction of 13, and the difference is which model authors the output. In 13 the cheap model writes the finished items and the strong model only deletes; deletion is a set of small yes/no decisions on completed work, and it went well. Here we tried the opposite handoff: the cheap model skims the text and passes a rough list of candidates to the strong model, which authors the final output while "focusing" on the shortlist. The strong model's internal reasoning quadrupled, because it weighed every candidate it was about to reject, and it anchored on the draft instead of reading the text fresh, more severely the more of the text the list covered. A rough draft contaminates the model that has to write over it; finished work merely gets filtered. The one part worth keeping: when the cheap model flagged rival readings of an ambiguous passage as explicit alternatives, the strong model picked the right one, fixing an error it had made when working alone.

4. **Turning thinking down raised output and cut cost by two thirds.** Reasoning models spend hidden "thinking" tokens before answering, and Gemini bills those at output rates: at our default setting, 77% of what we paid for was thought. Dialling thinking to minimal, against all intuition, made the model find more good items. The new errors it introduced fell in one recoverable class (cause and effect the wrong way round), fixable by a later pass, rather than invented content.

5. **One question beats twelve.** Our main prompt asks the model to decide many things per item at once: the claim, the labels, several metadata flags. A follow-up pass that revisited each finished item with a single question ("does this one flag apply to this item?") scored 100%, 14 of 14, including a

case the same model had got wrong when deciding everything at once. For reclassification jobs, a focused second look beats loading every decision into the first pass.

6. **Capacity starvation is per model.** When a cloud model is overloaded, requests fail with "resource exhausted" errors. We assumed a sibling model from the same family on the same route would be failing too, so it could not serve as a fallback. Measured during a real outage: the starved model failed 6 probes out of 6 while two same-family models on the same route served 6 out of 6. Capacity is provisioned per model, and a sibling is a valid fallback.

Pipeline design

1. **Chunk size dwarfed every prompt effect we measured.** How much text you send per request ("chunking") mattered more than anything we did to the prompt. Chunks of 2,000 characters found 50% more good items than 4,000. Fed a whole 89,000-character document in one request, the model returned 6 items where chunked runs returned 114: past a certain input length it stops extracting and summarises to a token budget, whatever the instructions say. Larger chunks are also less repeatable, with run-to-run overlap falling from 0.93 at 2,000 characters to 0.55 at 8,000. Extraction over a big context behaves like drawing a sample.

2. **Chunking loses the connections between chunks, and no amount of renaming buys them back.** The price of small chunks: the model can only report what it sees in one slice, so a cause discussed early in a document and its effect discussed pages later never come out as one claim, and instead of the joined-up network the user wants, the output arrives as scattered fragments. We tested every repair that works on the output alone, mostly passes that rename near-identical wordings so that claims about the same thing meet up ("staff morale" and "morale of the team" becoming one node). None of it reconnected anything, because the missing connections were claims that had never been extracted at all rather than matching claims under different names. What did work was a second pass over the whole text explicitly hunting for the missing connections. But such a pass finds real missed claims and also invents some, and the model does not reliably flag which is which, so every item it adds must be gated by a mechanical check in code (for us: the supporting quote must appear verbatim in the source, checked by string matching rather than by the model).

3. **Prompt caching was blocked three ways and worth little anyway.** Providers discount input tokens they have recently seen, which sounds like the fix when you re-send a long prompt with every chunk. For our Gemini pipeline it failed on three independent counts: our prompt was assembled with the varying part first, so there was no constant prefix to cache; Gemini's automatic caching never fired on the EU route we are required to use; and its explicit caching was not available EU-resident at all. Two of the three are provider-specific: Anthropic's caching works differently (you mark the cacheable prefix explicitly in the request), and when we used Claude for a separate feature it cached fine on the same EU setup. Only the first block, prompt assembled wrong way round, is universal, and it is the one to check first in any codebase. Caching also would not have paid here:

while thinking dominated the bill, perfect caching would have saved 21%. Shortening the prompt achieved the same end with no infrastructure.

Process

1. **Reasoning about why a prompt behaves as it does, from a handful of outputs, failed four times in one session.** Each time we read some output, formed a plausible mechanism, changed the prompt accordingly, and the change did nothing. What settled the question was a positive control: an old prompt known to produce the desired behaviour, run under exactly the current conditions. It localised the true cause quickly.

2. **A change that touches two modes must be measured in both.** Our product runs the same coding task in two output modes. We evaluated a new default thoroughly in one mode, shipped it to both, and silently broke the other. The metric that would have caught it already existed in our own tooling and was simply never looked at, because the runs that would have shown it were never made.

3. **External LLM judging needs a budget rule.** Automated judging via API might seem free per call, but routine judge runs cost $200 in two days. The standing rule now: a human (or the coding assistant, in this case Claude Code) judges by reading; a paid model judges only on explicit request.

Where it landed for us

Production default: gemini-3.5-flash, single pass, 2,000-character chunks, minimal-length core prompt composed from independent layers. And lots of findings specific to causal mapping. We are now moving that default towards larger chunks (16,000 characters) with the per-segment accounting of point 11, followed by the island-joining pass of point 19: together these keep the recall of small chunks while restoring the connected map that small chunks destroy.